

Space Complexity Radar

Made by @rathoreadiitya

Use this after writing or reading a solution. The goal is simple: identify extra memory used by your algorithm, check whether the input even fits, and state your answer clearly.

20-second scan

1. Check the constraint on n first. If $n \leq 10^9$, you cannot even store the input — the answer must be $O(1)$ or $O(\log n)$ space. If n fits comfortably ($\leq 10^7$), move on.
2. Most interviewers ask about auxiliary space — extra memory your algorithm uses on top of the input. That's the default unless told otherwise.
3. Count every extra data structure you create: hash maps, sets, stacks, queues, DP tables, copied arrays.
4. Count recursion stack depth. Recursive calls are not free — each frame sits on the call stack.
5. Count BFS queue or DFS stack size.
6. Output arrays are usually excluded unless the interviewer says "total space." If in doubt, ask.
7. If the interviewer says "total space complexity" or "including input," then add the input size too.

Space complexity = auxiliary (extra) memory used, unless interviewer says total.

Memory budget — what actually fits

Numbers below assume a **256 MB limit** (Codeforces, AtCoder, most ICPC, typical interview judges) and standard C++ types. $256 \text{ MB} \approx 2.68 \times 10^8 \text{ bytes}$.

C++ ceilings (heap allocation: vector, new, or globals — not stack)

Type	Hard ceiling	Safe to use	Notes
<code>int[]</code> (4 bytes)	$\sim 6 \times 10^7$	$\sim 10^7$	10^8 ints = 400 MB → MLE
<code>long long[]</code> or <code>double[]</code> (8 bytes)	$\sim 3 \times 10^7$	$\sim 10^7$	Half the count of int

bool[] or char[] (1 byte)	$\sim 2 \times 10^8$	$\sim 10^8$	vector<bool> is bit-packed, $\sim 8\times$ denser, slower
2D int matrix	$\sim 8000 \times 8000$	5000×5000	3000×3000 is totally safe
2D long long / double matrix	$\sim 5000 \times 5000$	3000×3000	Twice the cell cost of int matrix
unordered_map / unordered_set	$\sim 10^7$	$\sim 10^6$	Heavy node overhead. Don't push 10^8 .
map / set (red-black tree)	$\sim 10^7$	$\sim 10^6$	Slower than unordered, similar memory
Recursion depth	$\sim 10^6$	$\sim 10^5$	Online judges cap stack at 1–8 MB. Switch to iterative beyond 10^6 .
string of length n	$\sim 2 \times 10^8$	$\sim 10^8$	1 byte per char. Treat like char[].

If your judge gives 512 MB or 1024 MB (e.g. some LeetCode problems), double or quadruple every ceiling above.

Reverse rule — what n lets you do

Constraint on n	Memory you can afford	Why
$n \leq 10^9$	$O(1)$ or $O(\log n)$ only	Cannot even store the input. Math / two pointers / streaming.
$n \leq 10^7$	$O(n)$, $O(n \log n)$	Comfortably fits in 256 MB.
$n \leq 10^5$	$O(n)$, $O(n \log n)$ easy	Default sweet spot for most LeetCode hard.
$n \leq 10^4$	$O(n^2)$ cuts it close	10^8 ints = 400 MB $\rightarrow$ MLE on 256 MB. Only works if cells are bytes.
$n \leq 5000$	$O(n^2)$ int memory fine	2.5×10^7 ints $\approx$ 100 MB.
$n \leq 1000$	$O(n^2)$ comfortable	~ 4 MB. $O(n^3)$ memory not fine (10^9 ints).
$n \leq 300$	$O(n^3)$ memory fits	$\sim 1 \times 10^8$ ints. Use rolling array if tight.
$n \leq 200$	$O(n^3)$ totally safe	8×10^6 ints $\approx$ 32 MB.

Most used space signals

Start here. These are the space answers you will say most often.

Pattern	Spot it in code	Classic problem	Space
Only variables and pointers	int counters, two pointers l, r, swap	Reverse Linked List, Two Sum on sorted	$O(1)$

	temp	array	
Hash map / frequency map	HashMap, dict, Counter, unordered_map	Two Sum, Group Anagrams	O(n)
Hash set visited / seen	HashSet, set(), unordered_set	Contains Duplicate, Longest Substring Without Repeating	O(n)
Monotonic stack	Stack, list with append / pop	Daily Temperatures, Largest Rectangle in Histogram	O(n)
Queue for BFS	Queue, Deque, collections.deque	Rotting Oranges, Word Ladder	O(n)
DFS recursion on tree	recursive calls on node.left, node.right	Max Depth, Validate BST	O(h)
DFS recursion, skewed tree	same as above, tree degenerates to a list	Flatten Binary Tree to Linked List, pathological BST	O(n)
DP 1D table	dp = [0] * n, int[] dp = new int[n+1]	Climbing Stairs, House Robber, Coin Change	O(n)
DP 2D table	dp[i][j], [[0]*m for _ in range(n)]	Edit Distance, LCS, Unique Paths	O(n × m)
Sliding window with fixed counters	int[26] or a few int vars, no map	Find All Anagrams, Max Vowels in Substring	O(1)
Sliding window with map	HashMap storing chars / nums in window	Longest Substring Without Repeating, Min Window Substring	O(k) or O(n)
Heap of size k	PriorityQueue, heapq, pop when size > k	Kth Largest Element, Top K Frequent Elements	O(k)

Exhaustive space list

Use this when recursion, graph, tree, sorting, or output memory is confusing.

Pattern	Spot it in code	Classic problem	Space
Few variables	only int / bool / pointers	Reverse a number	O(1)
Two pointers	l = 0, r = n - 1, no extra DS	Valid Palindrome, 3Sum (after sort)	O(1)
Sorting in-place	Arrays.sort(), arr.sort(), no copy	Quicksort, in-place sort step of any algo	O(1) or O(log n)
Sorting with a copied array	sorted(arr) in Python, Arrays.copyOf	Any time you keep the original array intact	O(n)
Hash map	HashMap, dict, unordered_map	Two Sum, LRU Cache (map + DLL)	O(n)

Hash set	HashSet, set(), unordered_set	Longest Consecutive Sequence	O(n)
Frequency array of 26 chars	int[26], [0]*26	Valid Anagram, Find All Anagrams	O(1)
Frequency array of size n	int[n+1] for counting occurrences	Counting Sort, First Missing Positive	O(n)
Stack	Stack, Deque as stack, list.append/pop	Valid Parentheses, Evaluate RPN	O(n)
Queue BFS on a tree	BFS level traversal of a tree	Binary Tree Level Order Traversal	O(w) — max width
Queue BFS on a graph	BFS with visited set + queue	Number of Islands (BFS), Word Ladder	O(V)
DFS recursion on a tree	dfs(node.left); dfs(node.right);	Max Depth, Diameter of Binary Tree	O(h)
DFS recursion on a graph	recursive dfs(neighbor) + visited	Number of Islands (DFS), Course Schedule	O(V)
Grid visited matrix	boolean[][] visited = new boolean[r][c]	Number of Islands, Pacific Atlantic Water Flow	O(rows × cols)
Modify grid in-place	mark cells as '#' or 0 instead of using visited	Number of Islands variant that mutates input	O(1) extra
DP 1D	dp[i]	Climbing Stairs, House Robber	O(n)
DP 2D	dp[i][j]	Edit Distance, Unique Paths, 0/1 Knapsack	O(n × m)
Optimized DP rolling array	only prev and curr rows kept	0/1 Knapsack with 1D dp, Fibonacci with two vars	O(1) or O(m)
Backtracking recursion	path.append(x); recurse; path.pop()	Permutations, Subsets, N-Queens	O(depth)
Backtracking result list	result.append(list(path))	Subsets, Combinations	Output space (excluded by default)
Heap with all elements	heapify(arr), push all n items	Heap Sort, Last Stone Weight	O(n)
Heap of size k	push then pop when size exceeds k	Kth Largest Element, Top K Frequent	O(k)
Union-Find / DSU	parent[] and rank[]	Connected Components, Redundant Connection	O(n)
Prefix sum array	prefix[i+1] = prefix[i] + arr[i]	Range Sum Query, Subarray Sum Equals K	O(n)

Recursion with memo	@lru_cache, HashMap<State, Integer> memo	Word Break, Decode Ways, Top-down Knapsack	O(states)
Trie	TrieNode with children map/array	Implement Trie, Word Search II	O(total chars inserted)
Adjacency list	List<List<Integer>>, defaultdict(list)	Graph BFS/DFS, Course Schedule	O(V + E)
Segment tree / BIT	int[4*n] or int[n+1]	Range Sum Query Mutable, Count of Smaller Numbers	O(n)

Interview answer format

Use this:

Space complexity is $O(\dots)$, because ...

Example:

Space complexity is $O(n)$, because the hash map can store every element in the worst case.

Common traps

Wrong thinking	Correct thinking
"One loop means $O(1)$ space"	Time and space are separate. A single loop can still allocate a hash map of size n .
"Recursion is free"	Recursive calls live on the call stack and count as space.
"Input also counts"	Input is not counted unless the interviewer specifically asks.
"BFS queue is small"	BFS can store an entire level of the tree or graph at once.
"Sorting is always $O(1)$ space"	Depends on language and algorithm. Python's Timsort uses $O(n)$. Java's sort on objects uses $O(n)$.
"Output counts"	Output is excluded by default. Mention it only if asked.

Practice drill: for the next 5 solutions, write the data structures used, the recursion depth, and the final space complexity before checking the editorial.